

```

# =====
# 8. PCA FOR FEATURE REDUCTION
# Dimensionality Reduction of Extracted Image Features
# =====

# -----
# STEP 1: IMPORT LIBRARIES
# -----

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

from skimage.feature import hog

# -----
# STEP 2: LOAD IMAGE DATASET
# -----

digits = load_digits()

images = digits.images
labels = digits.target

print("Dataset loaded successfully")
print("Number of images:", len(images))
print("Image size:", images[0].shape)

# -----
# STEP 3: DISPLAY SAMPLE IMAGES
# -----

plt.figure(figsize=(10, 5))

for i in range(10):

    plt.subplot(2, 5, i + 1)

    plt.imshow(
        images[i],
        cmap="gray"
    )

    plt.title(
        "Label: " + str(labels[i])
    )

    plt.axis("off")

plt.tight_layout()
plt.show()

# -----
# STEP 4: EXTRACT HOG FEATURES

```

```
# STEP 4: EXTRACT HOG FEATURES
# -----

print("\nExtracting HOG features...")

hog_features = []

for image in images:

    features = hog(
        image,
        orientations=9,
        pixels_per_cell=(2, 2),
        cells_per_block=(2, 2),
        block_norm="L2-Hys"
    )

    hog_features.append(features)

X = np.array(hog_features)
y = labels

print("HOG feature extraction completed.")

print(
    "Original feature dimension:",
    X.shape[1]
)

print(
    "Number of samples:",
    X.shape[0]
)

# -----
# STEP 5: SPLIT DATA INTO TRAINING AND TESTING
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# -----
# STEP 6: STANDARDIZE FEATURES
# -----

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(
    X_train
)

X_test_scaled = scaler.transform(
    X_test
)

print("\nFeature standardization completed.")
```

```
# -----
# STEP 7: BASELINE WITHOUT PCA
# -----

print("\n=====")
print("      BASELINE - WITHOUT PCA")
print("=====")

start_time = time.perf_counter()

knn_original = KNeighborsClassifier(
    n_neighbors=5
)

knn_original.fit(
    X_train_scaled,
    y_train
)

train_time_original = (
    time.perf_counter() - start_time
)

start_time = time.perf_counter()

y_pred_original = knn_original.predict(
    X_test_scaled
)

test_time_original = (
    time.perf_counter() - start_time
)

accuracy_original = accuracy_score(
    y_test,
    y_pred_original
) * 100

print(
    "Original Dimensions:",
    X_train_scaled.shape[1]
)

print(
    "Training Time:",
    round(train_time_original, 4),
    "seconds"
)

print(
    "Testing Time:",
    round(test_time_original, 4),
    "seconds"
)

print(
    "Accuracy:",
    round(accuracy_original, 2),
    "%"
)

# -----
# STEP 8: APPLY PCA
# -----
```

```
print("\n=====")
print("          PCA ANALYSIS")
print("=====")

# PCA retaining 95% variance
pca_95 = PCA(
    n_components=0.95
)

X_train_pca_95 = pca_95.fit_transform(
    X_train_scaled
)

X_test_pca_95 = pca_95.transform(
    X_test_scaled
)

print(
    "Original dimensions:",
    X_train_scaled.shape[1]
)

print(
    "Reduced dimensions:",
    X_train_pca_95.shape[1]
)

print(
    "Variance retained:",
    round(
        np.sum(
            pca_95.explained_variance_ratio_
        ) * 100,
        2
    ),
    "%"
)

# -----
# STEP 9: KNN AFTER PCA
# -----

start_time = time.perf_counter()

knn_pca = KNeighborsClassifier(
    n_neighbors=5
)

knn_pca.fit(
    X_train_pca_95,
    y_train
)

train_time_pca = (
    time.perf_counter() - start_time
)

start_time = time.perf_counter()

y_pred_pca = knn_pca.predict(
    X_test_pca_95
)

test_time_pca = (
```

```
time.perf_counter() - start_time
)

accuracy_pca = accuracy_score(
    y_test,
    y_pred_pca
) * 100

print("\nPCA + KNN Results")

print(
    "Reduced dimensions:",
    X_train_pca_95.shape[1]
)

print(
    "Training Time:",
    round(train_time_pca, 4),
    "seconds"
)

print(
    "Testing Time:",
    round(test_time_pca, 4),
    "seconds"
)

print(
    "Accuracy:",
    round(accuracy_pca, 2),
    "%"
)

# -----
# STEP 10: TEST DIFFERENT PCA DIMENSIONS
# -----

pca_dimensions = [
    10,
    20,
    30,
    40,
    50,
    75,
    100
]

results = []

for n_components in pca_dimensions:

    # Make sure components do not exceed
    # the available feature dimensions

    if n_components >= X_train_scaled.shape[1]:
        continue

    # -----
    # PCA
    # -----

    pca = PCA(
        n_components=n_components
    )
```

```

start_time = time.perf_counter()

X_train_reduced = pca.fit_transform(
    X_train_scaled
)

X_test_reduced = pca.transform(
    X_test_scaled
)

pca_time = (
    time.perf_counter() - start_time
)

# -----
# KNN TRAINING
# -----

start_time = time.perf_counter()

knn = KNeighborsClassifier(
    n_neighbors=5
)

knn.fit(
    X_train_reduced,
    y_train
)

training_time = (
    time.perf_counter() - start_time
)

# -----
# PREDICTION
# -----

start_time = time.perf_counter()

predictions = knn.predict(
    X_test_reduced
)

testing_time = (
    time.perf_counter() - start_time
)

# -----
# ACCURACY
# -----

accuracy = accuracy_score(
    y_test,
    predictions
) * 100

# -----
# EXPLAINED VARIANCE
# -----

variance = (
    np.sum(
        pca.explained_variance_ratio_
    ) * 100
)

```

```

# -----
# STORE RESULTS
# -----

results.append([
    n_components,
    round(variance, 2),
    round(training_time, 5),
    round(testing_time, 5),
    round(accuracy, 2)
])

# -----
# STEP 11: CREATE RESULTS TABLE
# -----

results_df = pd.DataFrame(
    results,
    columns=[
        "PCA Components",
        "Explained Variance (%)",
        "Training Time (s)",
        "Testing Time (s)",
        "Accuracy (%)"
    ]
)

print("\n=====")
print("      PCA FEATURE REDUCTION RESULTS")
print("=====")

print(
    results_df.to_string(
        index=False
    )
)

# -----
# STEP 12: PCA EXPLAINED VARIANCE
# -----

pca_full = PCA()

pca_full.fit(
    X_train_scaled
)

cumulative_variance = np.cumsum(
    pca_full.explained_variance_ratio_
)

plt.figure(figsize=(10, 6))

plt.plot(
    range(
        1,
        len(cumulative_variance) + 1
    ),
    cumulative_variance * 100,
    linewidth=2
)

plt.axhline(

```

```

        y=95,
        linestyle="--",
        label="95% Variance"
    )

plt.xlabel(
    "Number of Principal Components"
)

plt.ylabel(
    "Cumulative Explained Variance (%)"
)

plt.title(
    "PCA Scree / Cumulative Variance Plot"
)

plt.legend()

plt.grid(True)

plt.tight_layout()

plt.show()

# -----
# STEP 13: ACCURACY VS PCA DIMENSION
# -----

plt.figure(figsize=(10, 6))

plt.plot(
    results_df["PCA Components"],
    results_df["Accuracy (%)"],
    marker="o",
    linewidth=2
)

plt.xlabel(
    "Number of PCA Components"
)

plt.ylabel(
    "Classification Accuracy (%)"
)

plt.title(
    "Accuracy vs PCA Feature Dimension"
)

plt.grid(True)

plt.tight_layout()

plt.show()

# -----
# STEP 14: TRAINING TIME COMPARISON
# -----

plt.figure(figsize=(10, 6))

plt.plot(
    results_df["PCA Components"],
    results_df["Training Time (s)"]

```



```
results_df["Training Time (s)"],
marker="o",
linewidth=2
)

plt.xlabel(
    "Number of PCA Components"
)

plt.ylabel(
    "Training Time (seconds)"
)

plt.title(
    "Computational Efficiency after PCA"
)

plt.grid(True)

plt.tight_layout()

plt.show()

# -----
# STEP 15: TESTING TIME COMPARISON
# -----

plt.figure(figsize=(10, 6))

plt.plot(
    results_df["PCA Components"],
    results_df["Testing Time (s)"],
    marker="o",
    linewidth=2
)

plt.xlabel(
    "Number of PCA Components"
)

plt.ylabel(
    "Testing Time (seconds)"
)

plt.title(
    "Testing Time vs PCA Components"
)

plt.grid(True)

plt.tight_layout()

plt.show()

# -----
# STEP 16: 2D PCA VISUALIZATION
# -----

pca_2d = PCA(
    n_components=2
)

X_2d = pca_2d.fit_transform(
    X_train_scaled
)
```

```

plt.figure(figsize=(10, 7))

scatter = plt.scatter(
    X_2d[:, 0],
    X_2d[:, 1],
    c=y_train,
    cmap="tab10",
    s=15
)

plt.xlabel(
    "Principal Component 1"
)

plt.ylabel(
    "Principal Component 2"
)

plt.title(
    "2D PCA Feature Representation"
)

plt.colorbar(
    scatter,
    label="Digit Class"
)

plt.grid(True)

plt.tight_layout()

plt.show()

# -----
# STEP 17: DIMENSION REDUCTION SUMMARY
# -----

original_dimension = X.shape[1]

reduced_dimension = X_train_pca_95.shape[1]

reduction_percentage = (
    1 -
    reduced_dimension /
    original_dimension
) * 100

print("\n=====")
print("          DIMENSION REDUCTION SUMMARY")
print("=====")

print(
    "Original feature dimension:",
    original_dimension
)

print(
    "Reduced feature dimension:",
    reduced_dimension
)

print(
    "Dimensionality reduction:",
    round(
        reduction_percentage

```

```

        2
    ),
    "%")
)

print(
    "Original accuracy:",
    round(
        accuracy_original,
        2
    ),
    "%")
)

print(
    "PCA accuracy:",
    round(
        accuracy_pca,
        2
    ),
    "%")
)

print(
    "Original training time:",
    round(
        train_time_original,
        4
    ),
    "seconds"
)

print(
    "PCA training time:",
    round(
        train_time_pca,
        4
    ),
    "seconds"
)

# -----
# STEP 18: SAVE RESULTS
# -----

results_df.to_csv(
    "PCA_feature_reduction_results.csv",
    index=False
)

print(
    "\nResults saved as:"
)

print(
    "PCA_feature_reduction_results.csv"
)

# -----
# STEP 19: FINAL OBSERVATION
# -----

print("\n=====")
print("OBSERVATION")

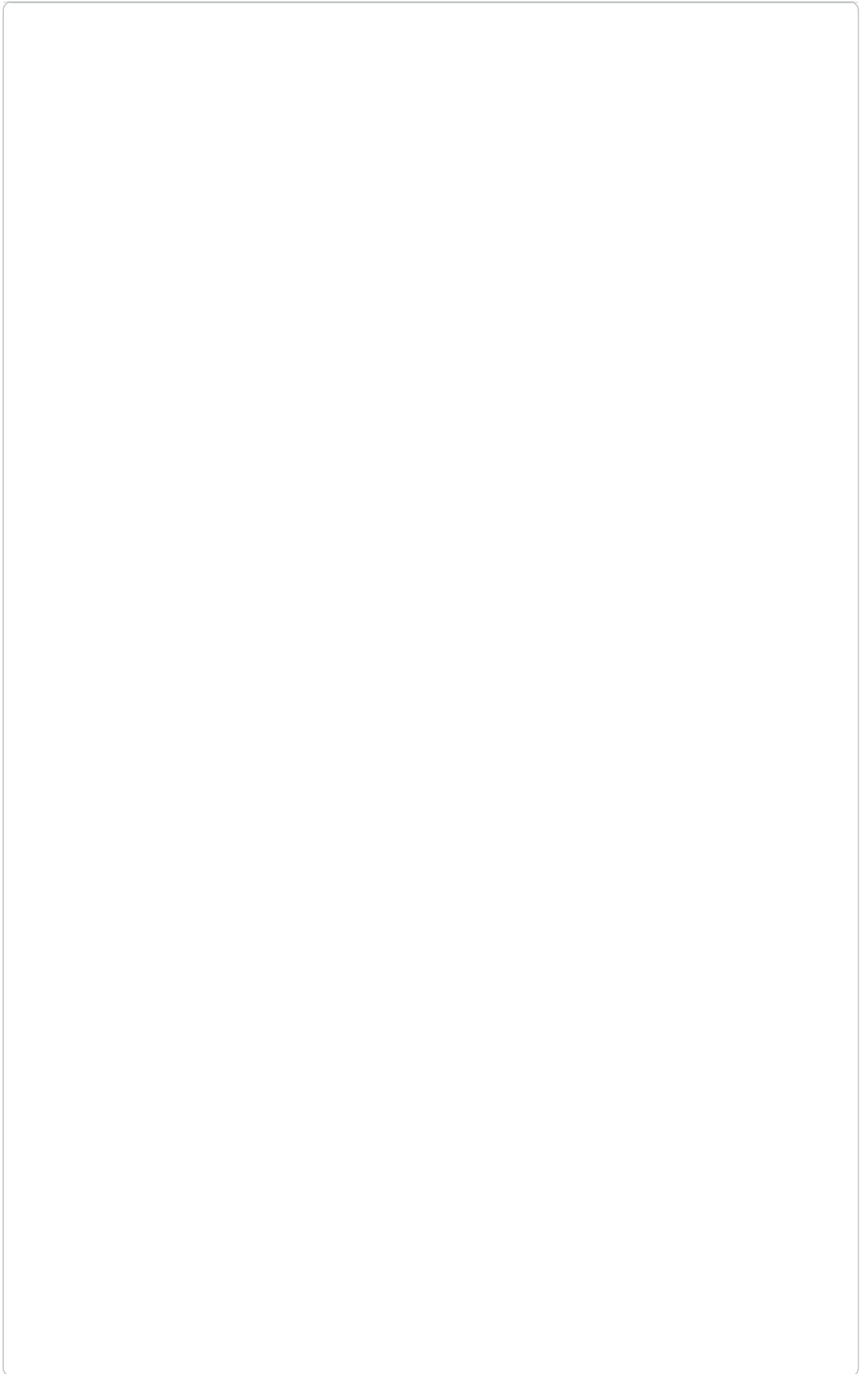
```

```
print("=====")

print("""
PCA successfully reduced the dimensionality of the
extracted HOG image features.

Therefore, an appropriate number of PCA components
should be selected to obtain a good balance between:

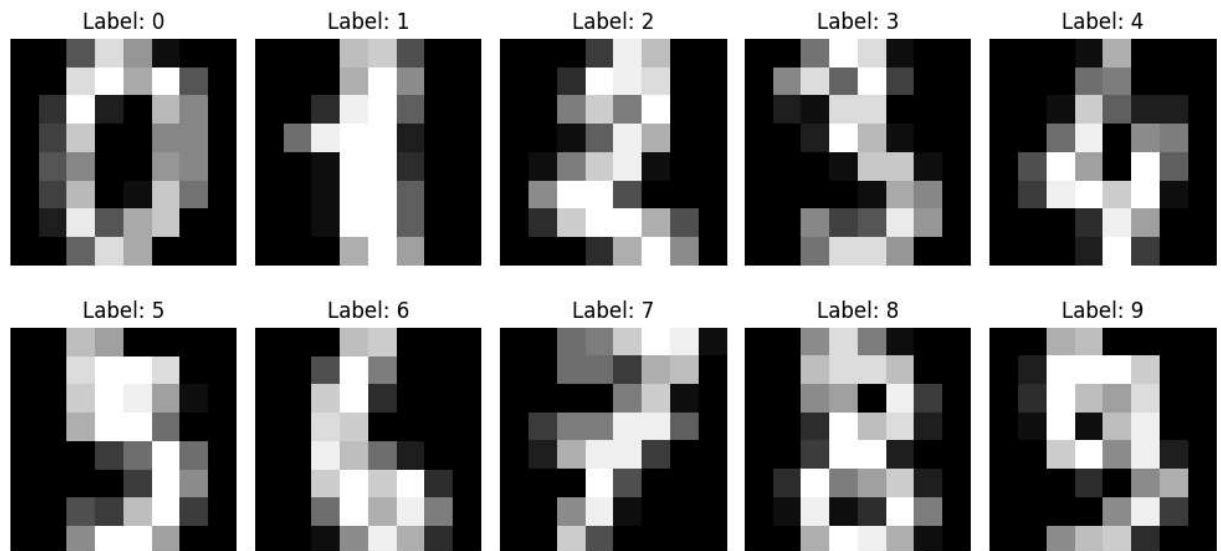
1. Feature representation quality
2. Computational efficiency
3. Classification accuracy
""")
```



Dataset loaded successfully

Number of images: 1797

Image size: (8, 8)



Extracting HOG features...

HOG feature extraction completed.

Original feature dimension: 324

Number of samples: 1797

Training samples: 1437

Testing samples: 360

Feature standardization completed.

=====

BASELINE - WITHOUT PCA

=====

Original Dimensions: 324

Training Time: 0.0021 seconds

Testing Time: 0.0249 seconds

Accuracy: 92.22 %

=====

PCA ANALYSIS

=====

Original dimensions: 324

Reduced dimensions: 109

Variance retained: 95.14 %

PCA + KNN Results

Reduced dimensions: 109

Training Time: 0.0031 seconds

Testing Time: 0.0162 seconds

Accuracy: 91.67 %

=====

PCA FEATURE REDUCTION RESULTS

=====

PCA Components	Explained Variance (%)	Training Time (s)	Testing Time (s)	Accuracy (%)
10	28.44	0.00355	0.01371	91.39
20	42.05	0.00134	0.00654	92.22
30	53.39	0.00408	0.01639	90.83
40	62.11	0.00345	0.01933	91.67
50	69.27	0.00355	0.01953	92.22
75	83.39	0.00167	0.02325	91.11
100	92.76	0.00377	0.02542	91.11

PCA Scree / Cumulative Variance Plot

